

PRACTICAL RECORD

DBMS Lab

Course Code: U25AMILPC307

Practical No. P4

*Multi-Table Join Queries – Implementing INNER, OUTER, SELF, and CROSS
JOIN on relational tables*

Name	Sagar Kumar
PRN	SOE25BTAM29
Section	A
Program	Bachelor of Technology
Year	2
Semester	III
Course Code	U25AMILPC307
Course Title	DBMS Lab
Practical No.	P4

Database	college_db (College Management System)
DBMS	PostgreSQL 17 (server 17.10)
Client tools	pgAdmin 4 (v9.18) Query Tool; psql 17

Date of Performance: _____

Date of Submission: _____

Marks: _____ / _____

Signature of Faculty: _____

Contents

No.	Section
1	Aim
2	Objectives
3	Theory
4	Database Schema
5	Join Concepts
6	SQL Queries
7	Actual Outputs (psql text + pgAdmin screenshots)
8	Explanation of Results
9	Result
10	Viva Questions and Answers
A	Appendix: Screenshot verification log

1. Aim

To write and execute multi-table join queries on the College Management System database (**college_db**) in PostgreSQL 17, implementing **INNER JOIN**, **LEFT OUTER JOIN**, **RIGHT OUTER JOIN**, **FULL OUTER JOIN**, **SELF JOIN** and **CROSS JOIN**, and joins involving three or more tables.

2. Objectives

- To understand how related data stored in separate normalised tables is combined using primary-key / foreign-key relationships.
- To implement INNER, LEFT, RIGHT and FULL OUTER joins and observe how each treats unmatched rows and NULL values.
- To use a SELF JOIN with table aliases to compare rows of the same table.
- To generate a Cartesian product with CROSS JOIN and verify that $|A \times B| = |A| \cdot |B|$.
- To write meaningful 3-, 4- and 5-table joins (student $\rightarrow$ enrollment $\rightarrow$ course $\rightarrow$ faculty $\rightarrow$ department) and combine joins with WHERE, GROUP BY and aggregate functions.
- To execute every query on a real PostgreSQL 17 server and record the actual output.

3. Theory

3.1 What is a Join?

In a relational database, data is split across several tables (normalisation) to avoid redundancy. A **join** is the relational operation that recombines rows from two or more tables into a single result, based on a related column – usually a **foreign key** in one table that references the **primary key** of another. In relational algebra a join is a Cartesian product followed by a selection:

$$R \bowtie_{\theta} S = \sigma_{\theta} (R \times S) \quad \text{e.g. } \text{student} \bowtie_{s.\text{department_id} = d.\text{department_id}} \text{department}$$

where θ is the join condition. If θ uses only equality ($=$) the join is an **equi-join**; if it uses other comparison operators ($<$, $>$, $<>$, ...) it is a **theta join**. A **natural join** is an equi-join on all columns having the same name, with duplicate columns removed.

3.2 General Syntax (ANSI SQL / PostgreSQL)

```
SELECT column_list
FROM   table1 [AS] t1
{ [INNER] JOIN | LEFT [OUTER] JOIN | RIGHT [OUTER] JOIN | FULL [OUTER] JOIN }
      table2 [AS] t2  ON t1.col = t2.col          -- or USING (col)
[ CROSS JOIN table3 ]                               -- no ON clause
[ WHERE condition ] [ GROUP BY ... ] [ ORDER BY ... ];
```

3.3 Types of Joins

- **INNER JOIN** – returns only those row combinations for which the join condition is TRUE. Unmatched rows of both tables are discarded. **JOIN** alone means INNER JOIN.
- **LEFT OUTER JOIN** – returns all rows of the left table; columns of the right table are NULL where there is no match.
- **RIGHT OUTER JOIN** – returns all rows of the right table; columns of the left table are NULL where there is no match. A **RIGHT JOIN B** $\equiv$ **B LEFT JOIN A**.
- **FULL OUTER JOIN** – returns matched rows plus unmatched rows from *both* tables (union of LEFT and RIGHT join results).
- **SELF JOIN** – a table joined with itself. It is not a separate keyword; it is an ordinary inner/outer join in which the same table appears twice under different **aliases**.
- **CROSS JOIN** – the Cartesian product: every row of the first table combined with every row of the second. It has no join condition; result size = rows(A) $\times$ rows(B).

3.4 Joins on More Than Two Tables

Joins are evaluated as a chain: the result of the first join is itself a (virtual) table that is joined with the next table. To connect n tables at least $n - 1$ join conditions are required; otherwise a Cartesian product is produced. A many-to-many relationship (student–course) is resolved through a bridge table (enrollment), so reaching course from student always passes through enrollment.

3.5 NULLs and Outer Joins

A comparison with NULL is never TRUE (it is UNKNOWN), so a row whose foreign key is NULL can never satisfy an equality join condition. Outer joins are therefore needed to keep such rows. Columns supplied by the missing side are filled with NULL and can be replaced for display using **COALESCE ()**. A condition on the optional table placed in the WHERE clause (instead of ON) turns an outer join back into an inner join.

3.6 How PostgreSQL Executes Joins

The PostgreSQL query planner chooses one of three physical join algorithms – **Nested Loop**, **Hash Join** or **Merge Join** – depending on table sizes, indexes and statistics. The chosen plan can be inspected with **EXPLAIN / EXPLAIN ANALYZE**. Indexes on join columns (primary keys are indexed automatically) make joins faster.

4. Database Schema

The practical uses the College Management System database **college_db**, consisting of five related tables. The complete creation and data script is provided as `sql/01_college_db_setup.sql`.

4.1 Relationships

Parent table (PK)	Child table (FK)	Cardinality	Meaning
department (department_id)	faculty (department_id)	1 : N	A department has many faculty members
department (department_id)	student (department_id)	1 : N	A department has many students (FK may be NULL)
department (department_id)	course (department_id)	1 : N	A department offers many courses
faculty (faculty_id)	course (faculty_id)	1 : N	A faculty member teaches courses (FK may be NULL)
student (student_id)	enrollment (student_id)	1 : N	Bridge table resolving student M:N course
course (course_id)	enrollment (course_id)	1 : N	

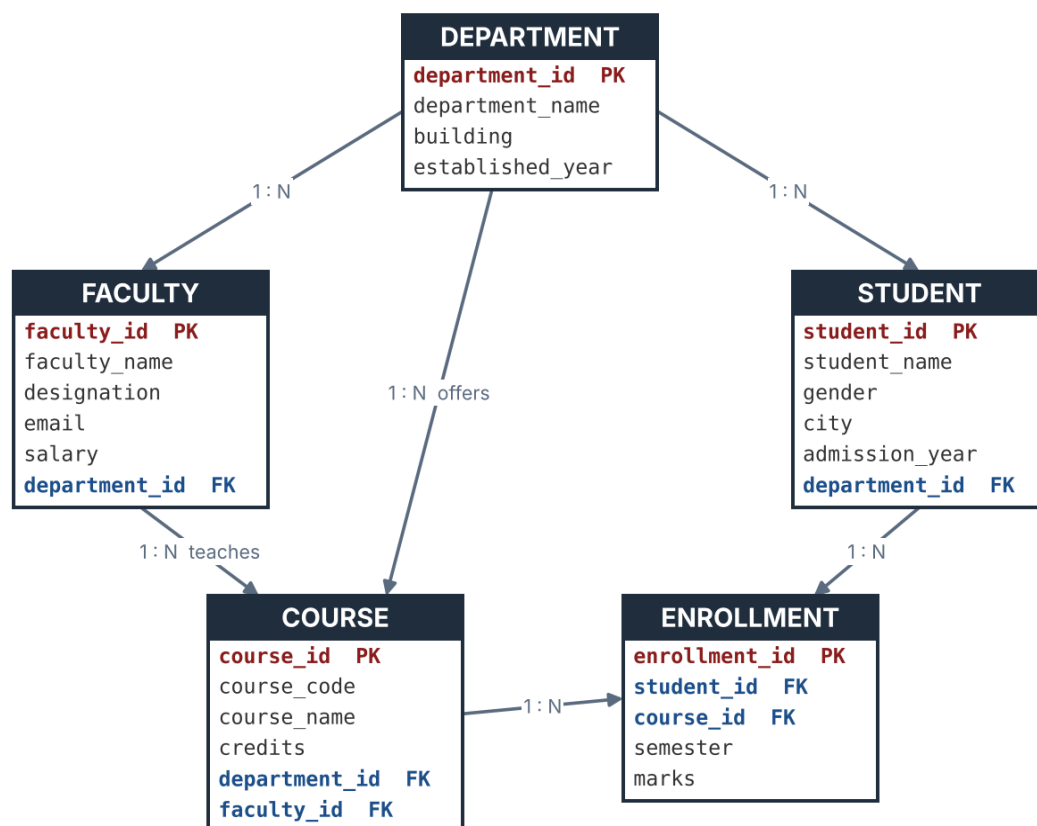


Figure B – Schema diagram of college_db (arrows point from the parent/PK side to the child/FK side)

4.2 Table Structures

(a) DEPARTMENT

Column	Data type	Constraint
department_id	INT	PRIMARY KEY
department_name	VARCHAR(60)	NOT NULL, UNIQUE
building	VARCHAR(30)	
established_year	INT	

(b) FACULTY

Column	Data type	Constraint
faculty_id	INT	PRIMARY KEY
faculty_name	VARCHAR(50)	NOT NULL
designation	VARCHAR(30)	
email	VARCHAR(60)	UNIQUE
salary	NUMERIC(10,2)	CHECK (salary > 0)
department_id	INT	FK → department(department_id)

(c) STUDENT

Column	Data type	Constraint
student_id	INT	PRIMARY KEY
student_name	VARCHAR(50)	NOT NULL
gender	CHAR(1)	CHECK (gender IN ('M','F'))
city	VARCHAR(30)	
admission_year	INT	
department_id	INT	FK → department; NULL = not yet allotted

(d) COURSE

Column	Data type	Constraint
course_id	INT	PRIMARY KEY
course_code	VARCHAR(15)	NOT NULL, UNIQUE
course_name	VARCHAR(60)	NOT NULL
credits	INT	CHECK (credits BETWEEN 1 AND 5)
department_id	INT	FK → department(department_id)
faculty_id	INT	FK → faculty; NULL = not yet assigned

(e) ENROLLMENT

Column	Data type	Constraint
enrollment_id	INT	PRIMARY KEY
student_id	INT	NOT NULL, FK → student
course_id	INT	NOT NULL, FK → course
semester	VARCHAR(5)	
marks	INT	CHECK (marks BETWEEN 0 AND 100)
(student_id, course_id)		UNIQUE

4.3 Sample Data (verbatim psql output of SELECT * FROM each table)

The data deliberately contains *unmatched* rows so that the difference between the join types becomes visible: two students without a department (9, 10), a department without faculty or students (50 – Civil Engineering), a course without a teacher (206 – Deep Learning), a faculty member without a course (107 – Prof. Kavita Rao) and two students without enrollments (9, 10).

SELECT * FROM department ORDER BY 1;

department_id	department_name	building	established_year
10	Artificial Intelligence & ML	Block A	2020
20	Computer Science	Block B	2008
30	Electronics & Telecom	Block C	2008
40	Mechanical Engineering	Block D	2010
50	Civil Engineering	Block E	2025

(5 rows)

SELECT * FROM faculty ORDER BY 1;

faculty_id	faculty_name	designation	email	salary	department_id
101	Dr. Anita Deshmukh	Professor	anita.d@college.edu	125000.00	10
102	Prof. Rahul Mehta	Assistant Professor	rahul.m@college.edu	78000.00	10
103	Dr. Sneha Kulkarni	Associate Professor	sneha.k@college.edu	98000.00	20
104	Prof. Vikram Joshi	Assistant Professor	vikram.j@college.edu	76000.00	20
105	Dr. Priya Nair	Professor	priya.n@college.edu	120000.00	30
106	Prof. Amit Patil	Assistant Professor	amit.p@college.edu	72000.00	40
107	Prof. Kavita Rao	Associate Professor	kavita.r@college.edu	95000.00	10

(7 rows)

SELECT * FROM student ORDER BY 1;

student_id	student_name	gender	city	admission_year	department_id
1	Aarav Sharma	M	Pune	2024	10
2	Diya Patel	F	Mumbai	2024	10
3	Rohan Verma	M	Nagpur	2024	20
4	Ishita Gupta	F	Pune	2024	20
5	Kabir Singh	M	Nashik	2024	30
6	Meera Iyer	F	Chennai	2024	10
7	Arjun Reddy	M	Hyderabad	2024	40
8	Sana Khan	F	Mumbai	2024	20
9	Neha Joshi	F	Pune	2025	(null)
10	Yash Kulkarni	M	Kolhapur	2025	(null)

(10 rows)

SELECT * FROM course ORDER BY 1;

course_id	course_code	course_name	credits	department_id	faculty_id
201	U25AMILPC307	DBMS Lab	1	10	102
202	U25AMPC301	Machine Learning	4	10	101
203	U25CSPC302	Data Structures	4	20	103
204	U25CSPC303	Operating Systems	3	20	104
205	U25ETPC301	Digital Electronics	3	30	105
206	U25AMPE304	Deep Learning	3	10	(null)
207	U25MEPC301	Thermodynamics	3	40	106

(7 rows)

SELECT * FROM enrollment ORDER BY 1;

enrollment_id	student_id	course_id	semester	marks
1	1	201	III	88
2	1	202	III	79
3	2	201	III	92
4	2	202	III	85
5	3	203	III	74
6	3	204	III	67
7	4	203	III	81
8	5	205	III	70
9	6	201	III	95
10	6	206	III	90
11	7	207	III	63
12	8	204	III	77
13	8	201	III	84

(13 rows)

5. Join Concepts

The shaded area in each diagram shows which rows appear in the result (A = left table, B = right table). These are illustrative diagrams, not screenshots.

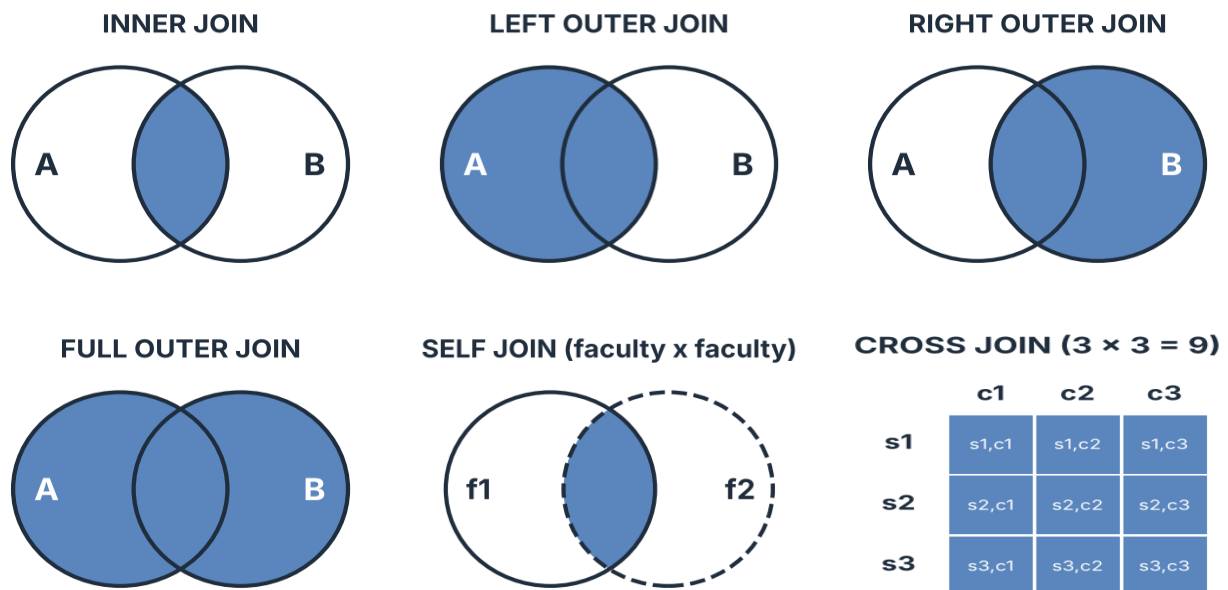


Figure A – Illustrative join diagrams

Join	Rows returned	Unmatched rows	Used in
INNER JOIN	Only matching pairs	Dropped from both tables	Q1, Q5, Q8, Q10
LEFT OUTER JOIN	All left rows + matches	Right-side columns = NULL	Q2, Q9, Q11, Q12
RIGHT OUTER JOIN	All right rows + matches	Left-side columns = NULL	Q3
FULL OUTER JOIN	All rows of both tables	NULL on the missing side	Q4
SELF JOIN	Pairs of rows from one table	Depends on inner/outer form	Q5
CROSS JOIN	$\text{rows}(A) \times \text{rows}(B)$	No condition, so none	Q6, Q7

Key points

- **INNER** and **OUTER** are optional keywords: **JOIN** = **INNER JOIN**, **LEFT JOIN** = **LEFT OUTER JOIN**.
- For an inner join, the order of tables does not change the result; for **LEFT**/**RIGHT** joins it does.
- Table aliases (**s**, **d**, **f1**, **f2** ...) shorten queries and are mandatory in a self join.
- In an outer join, conditions on the optional table belong in the **ON** clause; putting them in **WHERE** removes the **NULL**-extended rows.
- An anti-join (rows with no match) is written as **LEFT JOIN ... WHERE right.pk IS NULL**.

6. SQL Queries

All queries are stored in `sql/02_p4_join_queries.sql`. Each one was typed into the pgAdmin 4 Query Tool exactly as printed below and executed with F5 on PostgreSQL 17.

6.1 Q0: Environment check: server version and current database

Before running the joins, the server version and the connected database are confirmed. `version()` returns the PostgreSQL build string and `current_database()` returns the database the session is connected to.

SQL

```
-- Q0. Verify PostgreSQL server version and current database
SELECT version(), current_database();
```

6.2 Q1: INNER JOIN – Student + Department

Lists every student together with the name of his/her department. The **ON** clause matches `student.department_id` (foreign key) with `department.department_id` (primary key). Only rows having a match in *both* tables are returned.

SQL

```
-- Q1. INNER JOIN : students with their department
SELECT s.student_id, s.student_name, d.department_name
FROM student s
INNER JOIN department d
ON s.department_id = d.department_id
ORDER BY s.student_id;
```

6.3 Q2: LEFT OUTER JOIN – Student + Department

Same pair of tables, but every row of the *left* table (student) is preserved. Where no department matches, the department columns are filled with NULL.

SQL

```
-- Q2. LEFT OUTER JOIN : all students, department if allotted
SELECT s.student_id, s.student_name, d.department_name
FROM student s
LEFT OUTER JOIN department d
ON s.department_id = d.department_id
ORDER BY s.student_id;
```

6.4 Q3: RIGHT OUTER JOIN – Faculty + Department

Every row of the *right* table (department) is preserved, whether or not any faculty member works in it. Useful for spotting departments that still need staff.

SQL

```
-- Q3. RIGHT OUTER JOIN : all departments, with faculty if any
SELECT f.faculty_name, f.designation, d.department_name
FROM faculty f
RIGHT OUTER JOIN department d
ON f.department_id = d.department_id
ORDER BY d.department_id, f.faculty_id;
```

6.5 Q4: FULL OUTER JOIN – Course + Faculty

Returns matched course–faculty pairs *and* the unmatched rows from both sides: courses without a teacher and teachers without a course.

SQL

```
-- Q4. FULL OUTER JOIN : courses and faculty, matched or not
SELECT c.course_code, c.course_name, f.faculty_name
FROM course c
FULL OUTER JOIN faculty f
ON c.faculty_id = f.faculty_id
ORDER BY c.course_id, f.faculty_id;
```

6.6 Q5: SELF JOIN – Faculty members in the same department

The faculty table is joined with itself using two aliases, f1 and f2. Two rows pair up when they share the same department. The extra condition f1.faculty_id < f2.faculty_id removes a person paired with himself/herself and removes mirror duplicates (A,B)/(B,A). A third table, department, supplies the department name.

SQL

```
-- Q5. SELF JOIN : pairs of faculty in the same department
SELECT f1.faculty_name AS faculty_1,
       f2.faculty_name AS faculty_2,
       d.department_name
FROM faculty f1
INNER JOIN faculty f2
ON f1.department_id = f2.department_id
AND f1.faculty_id < f2.faculty_id
INNER JOIN department d
ON f1.department_id = d.department_id
ORDER BY d.department_name, f1.faculty_id, f2.faculty_id;
```

6.7 Q6: CROSS JOIN – Student × Course (AI & ML)

A CROSS JOIN has no join condition: it pairs every row of the first table with every row of the second (Cartesian product). Here it is restricted with WHERE to AI & ML students and AI & ML courses – a practical use is generating every possible registration combination.

SQL

```
-- Q6. CROSS JOIN : every AI&ML student paired with every AI&ML course
SELECT s.student_name, c.course_code, c.course_name
FROM student s
CROSS JOIN course c
WHERE s.department_id = 10
AND c.department_id = 10
ORDER BY s.student_id, c.course_id;
```

6.8 Q7: CROSS JOIN – size of the full Cartesian product

Counts the rows of the unrestricted `student` `CROSS JOIN` `course` and compares with the individual table sizes.

```
SQL
-- Q7. CROSS JOIN size = rows(student) x rows(course)
SELECT (SELECT COUNT(*) FROM student) AS students,
       (SELECT COUNT(*) FROM course)  AS courses,
       COUNT(*)                       AS cross_join_rows
FROM student CROSS JOIN course;
```

6.9 Q8: Three-table JOIN – Student → Enrollment → Course

enrollment is a bridge (associative) table that resolves the many-to-many relationship between student and course. Two join conditions are needed for three tables: student–enrollment on `student_id` and enrollment–course on `course_id`.

```
SQL
-- Q8. 3-table JOIN : student -> enrollment -> course
SELECT s.student_name, c.course_name, e.marks
FROM student s
INNER JOIN enrollment e ON s.student_id = e.student_id
INNER JOIN course c    ON e.course_id  = c.course_id
ORDER BY s.student_id, c.course_id;
```

6.10 Q9: Four-table JOIN – Student → Enrollment → Course → Faculty

Extends Q8 with the faculty who teaches each course. faculty is joined with a **LEFT JOIN** so that a course without an assigned teacher is not lost, and **COALESCE** replaces the NULL name with 'Not Assigned'.

```
SQL
-- Q9. 4-table JOIN : student -> enrollment -> course -> faculty
SELECT s.student_name, c.course_name,
       COALESCE(f.faculty_name, 'Not Assigned') AS faculty_name,
       e.marks
FROM student s
INNER JOIN enrollment e ON s.student_id = e.student_id
INNER JOIN course c    ON e.course_id  = c.course_id
LEFT JOIN faculty f    ON c.faculty_id = f.faculty_id
ORDER BY s.student_id, c.course_id;
```

6.11 Q10: Five-table JOIN – Toppers with department and faculty

Uses all five tables of college_db. It lists enrollments with marks ≥ 85 together with the student's department, the course and the teaching faculty, sorted by marks (highest first).

```
SQL
-- Q10. 5-table JOIN : toppers (marks >= 85) with department and faculty
SELECT s.student_name, d.department_name, c.course_name,
       f.faculty_name, e.marks
FROM student s
INNER JOIN department d ON s.department_id = d.department_id
INNER JOIN enrollment e ON s.student_id    = e.student_id
INNER JOIN course c    ON e.course_id      = c.course_id
INNER JOIN faculty f    ON c.faculty_id    = f.faculty_id
WHERE e.marks >= 85
ORDER BY e.marks DESC;
```

6.12 Q11: LEFT JOIN + IS NULL – Students not enrolled in any course (anti-join)

A LEFT JOIN keeps every student; students with no enrollment get NULL in all enrollment columns. Filtering on `e.enrollment_id IS NULL` keeps only those unmatched students.

SQL

```
-- Q11. LEFT JOIN + IS NULL : students not enrolled in any course
SELECT s.student_id, s.student_name, s.admission_year
FROM student s
LEFT JOIN enrollment e ON s.student_id = e.student_id
WHERE e.enrollment_id IS NULL
ORDER BY s.student_id;
```

6.13 Q12: Multi-table JOIN + GROUP BY – Department-wise summary

Starts from department and LEFT JOINS student and enrollment so that every department appears even if it has no students. Aggregate functions then count students and enrollments and average the marks per department.

SQL

```
-- Q12. Multi-table JOIN + GROUP BY : department-wise summary
SELECT d.department_name,
       COUNT(DISTINCT s.student_id) AS total_students,
       COUNT(e.enrollment_id)      AS enrollments,
       ROUND(AVG(e.marks), 2)      AS avg_marks
FROM department d
LEFT JOIN student s    ON d.department_id = s.department_id
LEFT JOIN enrollment e ON s.student_id    = e.student_id
GROUP BY d.department_id, d.department_name
ORDER BY d.department_id;
```

7. Actual Outputs

Authenticity of outputs and screenshots

Every output in this section comes from **real execution** on a PostgreSQL 17.10 server, database `college_db`. Each query has two records:

- **psql text output:** copied verbatim from the psql 17 client (NULL shown as `(null)`).
- **ACTUAL SCREENSHOT:** a screen capture of the pgAdmin 4 (v9.18) Query Tool after the query was run with F5 (NULL shown as `[null]`). No screenshot is simulated, edited or drawn.
- Cross-check: for every screenshot, the text in the pgAdmin editor was compared with the SQL in Section 6, and the result-grid cells were compared cell-by-cell with the psql output. All 13 matched (Appendix A).

No placeholders were needed: all 13 screenshots are actual captures.

7.1 Q0: Environment check: server version and current database

SQL EXECUTED

```
-- Q0. Verify PostgreSQL server version and current database  
SELECT version(), current_database();
```

ACTUAL OUTPUT – psql 17 (text)

version	current_database
PostgreSQL 17.10 on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 7.5.0-3ubuntu1~18.04) 7.5.0, 64-bit	college_db

(1 row)

ACTUAL SCREENSHOT • Figure 1

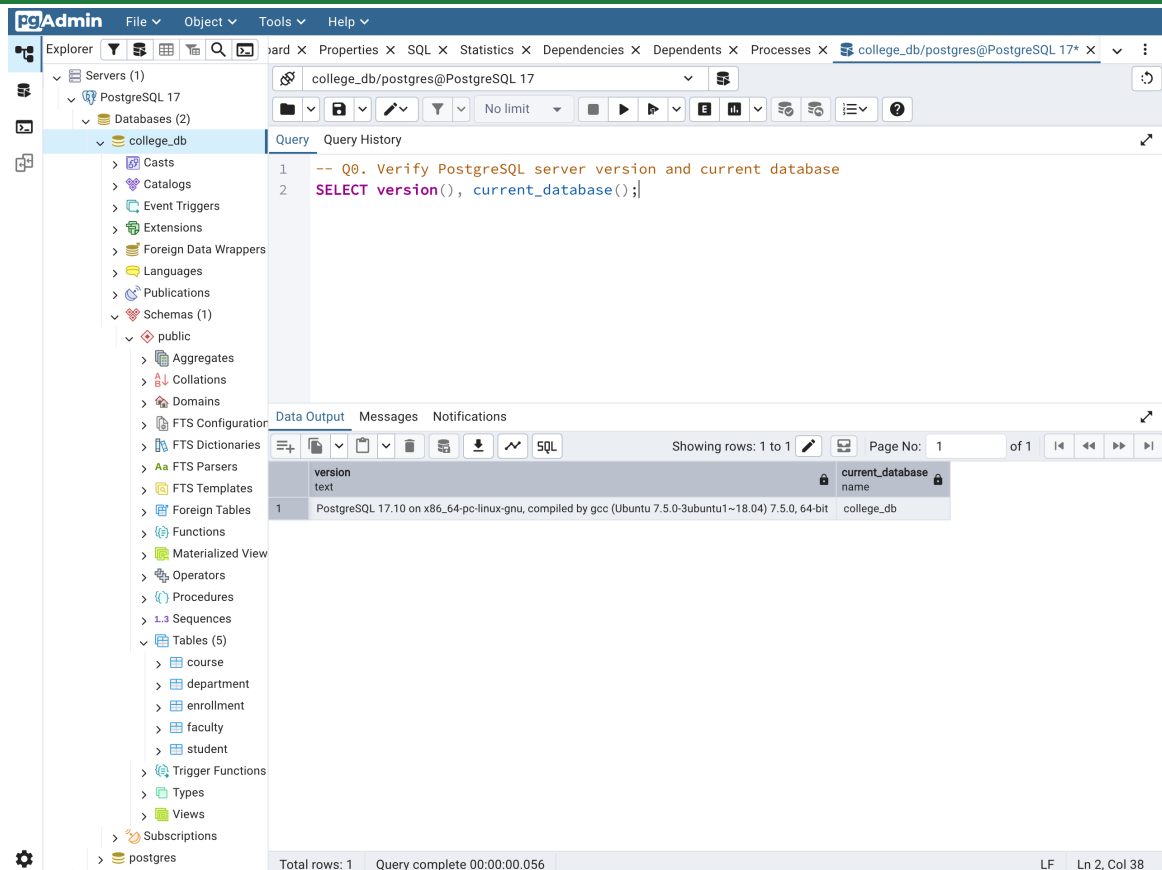


Figure 1 – Q0 executed in pgAdmin 4 Query Tool, connection college_db/postgres@PostgreSQL 17. pgAdmin status bar: “Total rows: 1 Query complete 00:00:00.056”. Rows returned: 1.

7.2 Q1: INNER JOIN – Student + Department

SQL EXECUTED

```
-- Q1. INNER JOIN : students with their department
SELECT s.student_id, s.student_name, d.department_name
FROM student s
INNER JOIN department d
ON s.department_id = d.department_id
ORDER BY s.student_id;
```

ACTUAL OUTPUT – psql 17 (text)

student_id	student_name	department_name
1	Aarav Sharma	Artificial Intelligence & ML
2	Diya Patel	Artificial Intelligence & ML
3	Rohan Verma	Computer Science
4	Ishita Gupta	Computer Science
5	Kabir Singh	Electronics & Telecom
6	Meera Iyer	Artificial Intelligence & ML
7	Arjun Reddy	Mechanical Engineering
8	Sana Khan	Computer Science

(8 rows)

ACTUAL SCREENSHOT • Figure 2

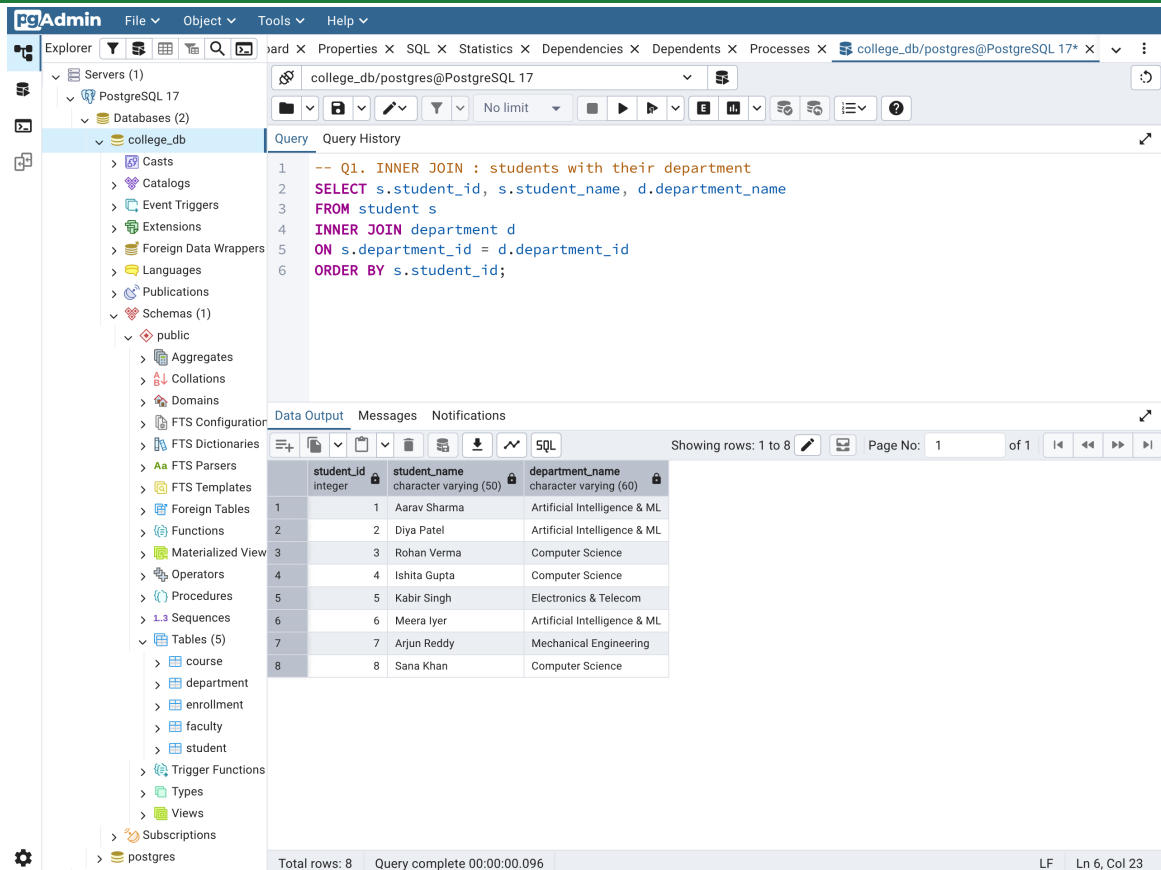


Figure 2 – Q1 executed in pgAdmin 4 Query Tool, connection college_db/postgres@PostgreSQL 17. pgAdmin status bar: “Total rows: 8 Query complete 00:00:00.096”. Rows returned: 8.

7.3 Q2: LEFT OUTER JOIN – Student + Department

SQL EXECUTED

```
-- Q2. LEFT OUTER JOIN : all students, department if allotted
SELECT s.student_id, s.student_name, d.department_name
FROM student s
LEFT OUTER JOIN department d
ON s.department_id = d.department_id
ORDER BY s.student_id;
```

ACTUAL OUTPUT – psql 17 (text)

student_id	student_name	department_name
1	Aarav Sharma	Artificial Intelligence & ML
2	Diya Patel	Artificial Intelligence & ML
3	Rohan Verma	Computer Science
4	Ishita Gupta	Computer Science
5	Kabir Singh	Electronics & Telecom
6	Meera Iyer	Artificial Intelligence & ML
7	Arjun Reddy	Mechanical Engineering
8	Sana Khan	Computer Science
9	Neha Joshi	(null)
10	Yash Kulkarni	(null)

(10 rows)

ACTUAL SCREENSHOT • Figure 3

The screenshot displays the pgAdmin 4 interface. The left pane shows the database structure with 'college_db' selected. The central pane shows the SQL query executed. The bottom pane shows the query results in a table format.

Query Executed:

```
-- Q2. LEFT OUTER JOIN : all students, department if allotted
SELECT s.student_id, s.student_name, d.department_name
FROM student s
LEFT OUTER JOIN department d
ON s.department_id = d.department_id
ORDER BY s.student_id;
```

Query Results:

student_id	student_name	department_name
1	Aarav Sharma	Artificial Intelligence & ML
2	Diya Patel	Artificial Intelligence & ML
3	Rohan Verma	Computer Science
4	Ishita Gupta	Computer Science
5	Kabir Singh	Electronics & Telecom
6	Meera Iyer	Artificial Intelligence & ML
7	Arjun Reddy	Mechanical Engineering
8	Sana Khan	Computer Science
9	Neha Joshi	[null]
10	Yash Kulkarni	[null]

Total rows: 10 Query complete 00:00:00.088

Figure 3 – Q2 executed in pgAdmin 4 Query Tool, connection college_db/postgres@PostgreSQL 17. pgAdmin status bar: “Total rows: 10 Query complete 00:00:00.088”. Rows returned: 10.

7.4 Q3: RIGHT OUTER JOIN – Faculty + Department

SQL EXECUTED

```
-- Q3. RIGHT OUTER JOIN : all departments, with faculty if any
SELECT f.faculty_name, f.designation, d.department_name
FROM faculty f
RIGHT OUTER JOIN department d
ON f.department_id = d.department_id
ORDER BY d.department_id, f.faculty_id;
```

ACTUAL OUTPUT – psql 17 (text)

faculty_name	designation	department_name
Dr. Anita Deshmukh	Professor	Artificial Intelligence & ML
Prof. Rahul Mehta	Assistant Professor	Artificial Intelligence & ML
Prof. Kavita Rao	Associate Professor	Artificial Intelligence & ML
Dr. Sneha Kulkarni	Associate Professor	Computer Science
Prof. Vikram Joshi	Assistant Professor	Computer Science
Dr. Priya Nair	Professor	Electronics & Telecom
Prof. Amit Patil	Assistant Professor	Mechanical Engineering
(null)	(null)	Civil Engineering

(8 rows)

ACTUAL SCREENSHOT • Figure 4

The screenshot shows the pgAdmin 4 interface with the following details:

- Query Editor:** Contains the SQL query for a RIGHT OUTER JOIN between the `faculty` and `department` tables, ordered by `department_id` and `faculty_id`.
- Data Output:** Displays the results of the query in a table with 8 rows. The columns are `faculty_name`, `designation`, and `department_name`.
- Status Bar:** Shows 'Total rows: 8' and 'Query complete 00:00:00.078'.

faculty_name	designation	department_name
Dr. Anita Deshmukh	Professor	Artificial Intelligence & ML
Prof. Rahul Mehta	Assistant Professor	Artificial Intelligence & ML
Prof. Kavita Rao	Associate Professor	Artificial Intelligence & ML
Dr. Sneha Kulkarni	Associate Professor	Computer Science
Prof. Vikram Joshi	Assistant Professor	Computer Science
Dr. Priya Nair	Professor	Electronics & Telecom
Prof. Amit Patil	Assistant Professor	Mechanical Engineering
[null]	[null]	Civil Engineering

Figure 4 – Q3 executed in pgAdmin 4 Query Tool, connection college_db/postgres@PostgreSQL 17. pgAdmin status bar: “Total rows: 8 Query complete 00:00:00.078”. Rows returned: 8.

7.5 Q4: FULL OUTER JOIN – Course + Faculty

SQL EXECUTED

```
-- Q4. FULL OUTER JOIN : courses and faculty, matched or not
SELECT c.course_code, c.course_name, f.faculty_name
FROM course c
FULL OUTER JOIN faculty f
ON c.faculty_id = f.faculty_id
ORDER BY c.course_id, f.faculty_id;
```

ACTUAL OUTPUT – psql 17 (text)

course_code	course_name	faculty_name
U25AMILPC307	DBMS Lab	Prof. Rahul Mehta
U25AMPC301	Machine Learning	Dr. Anita Deshmukh
U25CSPC302	Data Structures	Dr. Sneha Kulkarni
U25CSPC303	Operating Systems	Prof. Vikram Joshi
U25ETPC301	Digital Electronics	Dr. Priya Nair
U25AMPE304	Deep Learning	(null)
U25MEPC301	Thermodynamics	Prof. Amit Patil
(null)	(null)	Prof. Kavita Rao

(8 rows)

ACTUAL SCREENSHOT • Figure 5

The screenshot shows the pgAdmin 4 interface. The Query Editor contains the following SQL query:

```
-- Q4. FULL OUTER JOIN : courses and faculty, matched or not
SELECT c.course_code, c.course_name, f.faculty_name
FROM course c
FULL OUTER JOIN faculty f
ON c.faculty_id = f.faculty_id
ORDER BY c.course_id, f.faculty_id;
```

The Data Output pane displays the results of the query. The results are as follows:

course_code	course_name	faculty_name
U25AMILPC307	DBMS Lab	Prof. Rahul Mehta
U25AMPC301	Machine Learning	Dr. Anita Deshmukh
U25CSPC302	Data Structures	Dr. Sneha Kulkarni
U25CSPC303	Operating Systems	Prof. Vikram Joshi
U25ETPC301	Digital Electronics	Dr. Priya Nair
U25AMPE304	Deep Learning	[null]
U25MEPC301	Thermodynamics	Prof. Amit Patil
[null]	[null]	Prof. Kavita Rao

The status bar at the bottom indicates: Total rows: 8 Query complete 00:00:00.074

Figure 5 – Q4 executed in pgAdmin 4 Query Tool, connection college_db/postgres@PostgreSQL 17. pgAdmin status bar: “Total rows: 8 Query complete 00:00:00.074”. Rows returned: 8.

7.6 Q5: SELF JOIN – Faculty members in the same department

SQL EXECUTED

```
-- Q5. SELF JOIN : pairs of faculty in the same department
SELECT f1.faculty_name AS faculty_1,
       f2.faculty_name AS faculty_2,
       d.department_name
FROM faculty f1
INNER JOIN faculty f2
ON f1.department_id = f2.department_id
AND f1.faculty_id < f2.faculty_id
INNER JOIN department d
ON f1.department_id = d.department_id
ORDER BY d.department_name, f1.faculty_id, f2.faculty_id;
```

ACTUAL OUTPUT – psql 17 (text)

faculty_1	faculty_2	department_name
Dr. Anita Deshmukh	Prof. Rahul Mehta	Artificial Intelligence & ML
Dr. Anita Deshmukh	Prof. Kavita Rao	Artificial Intelligence & ML
Prof. Rahul Mehta	Prof. Kavita Rao	Artificial Intelligence & ML
Dr. Sneha Kulkarni	Prof. Vikram Joshi	Computer Science

(4 rows)

ACTUAL SCREENSHOT • Figure 6

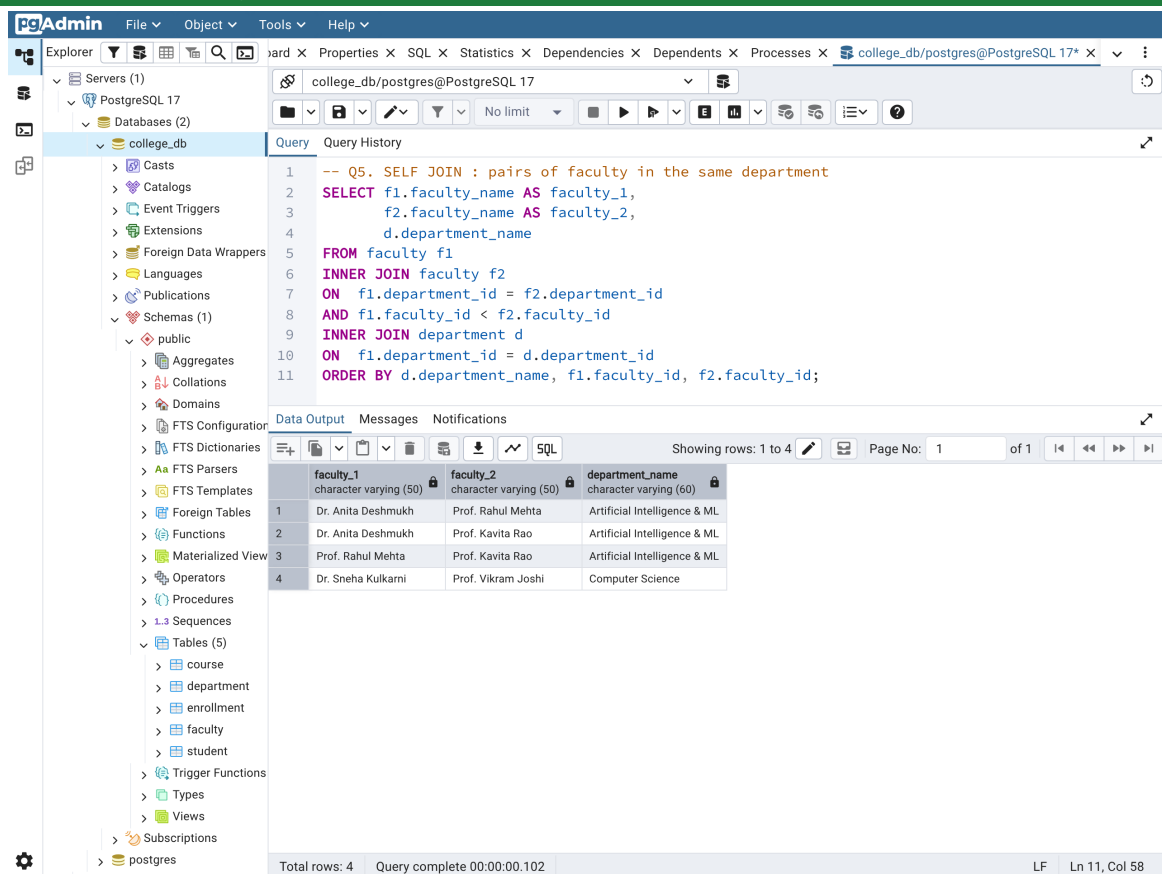


Figure 6 – Q5 executed in pgAdmin 4 Query Tool, connection college_db/postgres@PostgreSQL 17. pgAdmin status bar: “Total rows: 4 Query complete 00:00:00.102”. Rows returned: 4.

7.7 Q6: CROSS JOIN – Student × Course (AI & ML)

SQL EXECUTED

```
-- Q6. CROSS JOIN : every AI&ML student paired with every AI&ML course
SELECT s.student_name, c.course_code, c.course_name
FROM student s
CROSS JOIN course c
WHERE s.department_id = 10
AND c.department_id = 10
ORDER BY s.student_id, c.course_id;
```

ACTUAL OUTPUT – psql 17 (text)

student_name	course_code	course_name
Aarav Sharma	U25AMILPC307	DBMS Lab
Aarav Sharma	U25AMPC301	Machine Learning
Aarav Sharma	U25AMPE304	Deep Learning
Diya Patel	U25AMILPC307	DBMS Lab
Diya Patel	U25AMPC301	Machine Learning
Diya Patel	U25AMPE304	Deep Learning
Meera Iyer	U25AMILPC307	DBMS Lab
Meera Iyer	U25AMPC301	Machine Learning
Meera Iyer	U25AMPE304	Deep Learning

(9 rows)

ACTUAL SCREENSHOT • Figure 7

The screenshot displays the pgAdmin 4 interface. The Query Editor shows the following SQL query:

```
-- Q6. CROSS JOIN : every AI&ML student paired with every AI&ML course
SELECT s.student_name, c.course_code, c.course_name
FROM student s
CROSS JOIN course c
WHERE s.department_id = 10
AND c.department_id = 10
ORDER BY s.student_id, c.course_id;
```

The Data Output tab shows the following results:

student_name	course_code	course_name
Aarav Sharma	U25AMILPC307	DBMS Lab
Aarav Sharma	U25AMPC301	Machine Learning
Aarav Sharma	U25AMPE304	Deep Learning
Diya Patel	U25AMILPC307	DBMS Lab
Diya Patel	U25AMPC301	Machine Learning
Diya Patel	U25AMPE304	Deep Learning
Meera Iyer	U25AMILPC307	DBMS Lab
Meera Iyer	U25AMPC301	Machine Learning
Meera Iyer	U25AMPE304	Deep Learning

The status bar at the bottom indicates: Total rows: 9 Query complete 00:00:00.068 LF Ln 7, Col 36

Figure 7 – Q6 executed in pgAdmin 4 Query Tool, connection college_db/postgres@PostgreSQL 17. pgAdmin status bar: “Total rows: 9 Query complete 00:00:00.068”. Rows returned: 9.

7.8 Q7: CROSS JOIN – size of the full Cartesian product

SQL EXECUTED

```
-- Q7. CROSS JOIN size = rows(student) x rows(course)
SELECT (SELECT COUNT(*) FROM student) AS students,
       (SELECT COUNT(*) FROM course)  AS courses,
       COUNT(*)                        AS cross_join_rows
FROM student CROSS JOIN course;
```

ACTUAL OUTPUT – psql 17 (text)

```
students | courses | cross_join_rows
-----+-----+-----
      10 |       7 |             70
(1 row)
```

ACTUAL SCREENSHOT • Figure 8

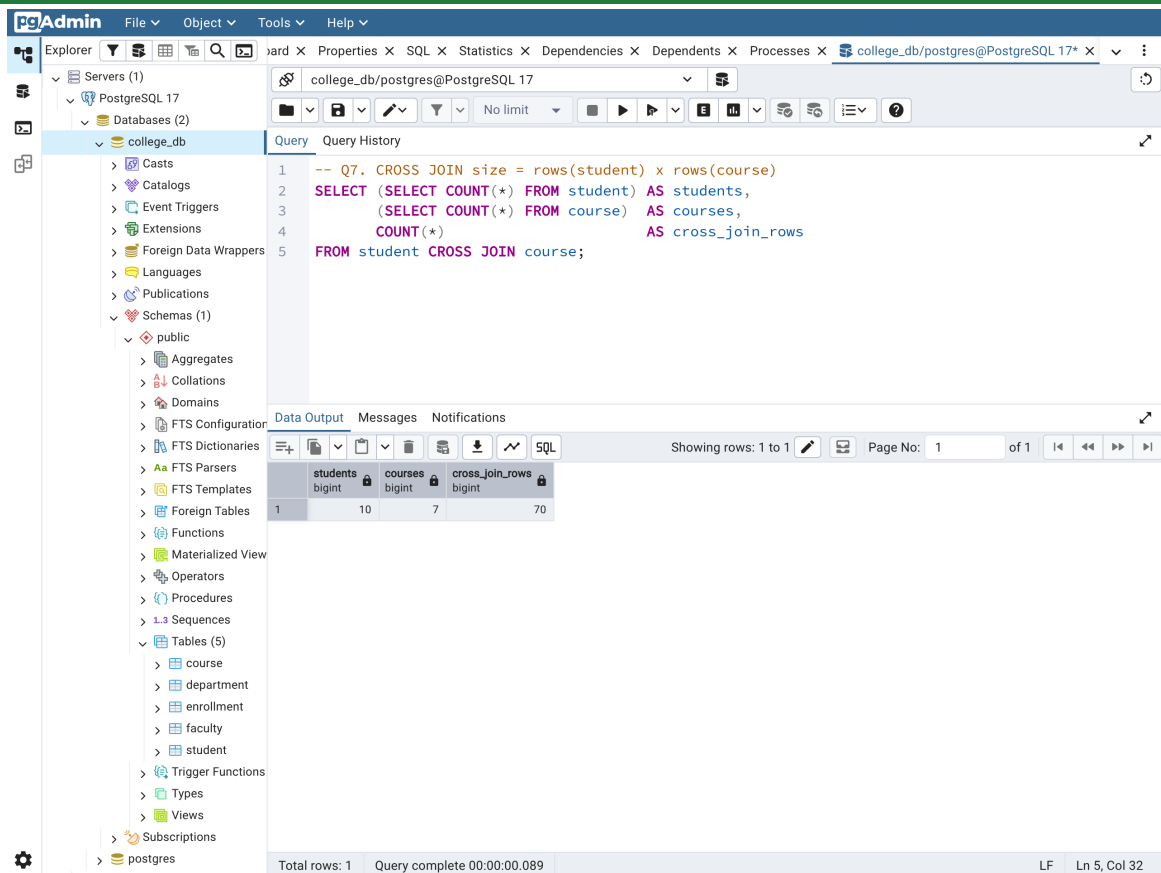


Figure 8 – Q7 executed in pgAdmin 4 Query Tool, connection college_db/postgres@PostgreSQL 17. pgAdmin status bar: “Total rows: 1 Query complete 00:00:00.089”. Rows returned: 1.

7.9 Q8: Three-table JOIN – Student → Enrollment → Course

SQL EXECUTED

```
-- Q8. 3-table JOIN : student -> enrollment -> course
SELECT s.student_name, c.course_name, e.marks
FROM student s
INNER JOIN enrollment e ON s.student_id = e.student_id
INNER JOIN course c      ON e.course_id = c.course_id
ORDER BY s.student_id, c.course_id;
```

ACTUAL OUTPUT – psql 17 (text)

student_name	course_name	marks
Aarav Sharma	DBMS Lab	88
Aarav Sharma	Machine Learning	79
Diya Patel	DBMS Lab	92
Diya Patel	Machine Learning	85
Rohan Verma	Data Structures	74
Rohan Verma	Operating Systems	67
Ishita Gupta	Data Structures	81
Kabir Singh	Digital Electronics	70
Meera Iyer	DBMS Lab	95
Meera Iyer	Deep Learning	90
Arjun Reddy	Thermodynamics	63
Sana Khan	DBMS Lab	84
Sana Khan	Operating Systems	77

(13 rows)

ACTUAL SCREENSHOT • Figure 9

The screenshot shows the pgAdmin 4 interface with the following details:

- Query Editor:** Contains the SQL query for a 3-table JOIN between student, enrollment, and course tables.
- Data Output:** Displays the results of the query in a table with 3 columns: student_name, course_name, and marks. There are 13 rows of data.
- Status Bar:** Shows 'Total rows: 13' and 'Query complete 00:00:00.066'.

student_name	course_name	marks
Aarav Sharma	DBMS Lab	88
Aarav Sharma	Machine Learning	79
Diya Patel	DBMS Lab	92
Diya Patel	Machine Learning	85
Rohan Verma	Data Structures	74
Rohan Verma	Operating Systems	67
Ishita Gupta	Data Structures	81
Kabir Singh	Digital Electronics	70
Meera Iyer	DBMS Lab	95
Meera Iyer	Deep Learning	90
Arjun Reddy	Thermodynamics	63
Sana Khan	DBMS Lab	84
Sana Khan	Operating Systems	77

Figure 9 – Q8 executed in pgAdmin 4 Query Tool, connection college_db/postgres@PostgreSQL 17. pgAdmin status bar: “Total rows: 13 Query complete 00:00:00.066”. Rows returned: 13.

7.10 Q9: Four-table JOIN – Student → Enrollment → Course → Faculty

SQL EXECUTED

```
-- Q9. 4-table JOIN : student -> enrollment -> course -> faculty
SELECT s.student_name, c.course_name,
       COALESCE(f.faculty_name, 'Not Assigned') AS faculty_name,
       e.marks
FROM student s
INNER JOIN enrollment e ON s.student_id = e.student_id
INNER JOIN course c     ON e.course_id = c.course_id
LEFT JOIN faculty f     ON c.faculty_id = f.faculty_id
ORDER BY s.student_id, c.course_id;
```

ACTUAL OUTPUT – psql 17 (text)

student_name	course_name	faculty_name	marks
Aarav Sharma	DBMS Lab	Prof. Rahul Mehta	88
Aarav Sharma	Machine Learning	Dr. Anita Deshmukh	79
Diya Patel	DBMS Lab	Prof. Rahul Mehta	92
Diya Patel	Machine Learning	Dr. Anita Deshmukh	85
Rohan Verma	Data Structures	Dr. Sneha Kulkarni	74
Rohan Verma	Operating Systems	Prof. Vikram Joshi	67
Ishita Gupta	Data Structures	Dr. Sneha Kulkarni	81
Kabir Singh	Digital Electronics	Dr. Priya Nair	70
Meera Iyer	DBMS Lab	Prof. Rahul Mehta	95
Meera Iyer	Deep Learning	Not Assigned	90
Arjun Reddy	Thermodynamics	Prof. Amit Patil	63
Sana Khan	DBMS Lab	Prof. Rahul Mehta	84
Sana Khan	Operating Systems	Prof. Vikram Joshi	77

(13 rows)

ACTUAL SCREENSHOT • Figure 10

The screenshot shows the pgAdmin 4 interface. The Query editor contains the following SQL query:

```
-- Q9. 4-table JOIN : student -> enrollment -> course -> faculty
SELECT s.student_name, c.course_name,
       COALESCE(f.faculty_name, 'Not Assigned') AS faculty_name,
       e.marks
FROM student s
INNER JOIN enrollment e ON s.student_id = e.student_id
INNER JOIN course c     ON e.course_id = c.course_id
LEFT JOIN faculty f     ON c.faculty_id = f.faculty_id
ORDER BY s.student_id, c.course_id;
```

The Data Output pane displays the results of the query in a table with the following columns: student_name, course_name, faculty_name, and marks. The results are as follows:

student_name	course_name	faculty_name	marks
Aarav Sharma	DBMS Lab	Prof. Rahul Mehta	88
Aarav Sharma	Machine Learning	Dr. Anita Deshmukh	79
Diya Patel	DBMS Lab	Prof. Rahul Mehta	92
Diya Patel	Machine Learning	Dr. Anita Deshmukh	85
Rohan Verma	Data Structures	Dr. Sneha Kulkarni	74
Rohan Verma	Operating Systems	Prof. Vikram Joshi	67
Ishita Gupta	Data Structures	Dr. Sneha Kulkarni	81
Kabir Singh	Digital Electronics	Dr. Priya Nair	70
Meera Iyer	DBMS Lab	Prof. Rahul Mehta	95
Meera Iyer	Deep Learning	Not Assigned	90
Arjun Reddy	Thermodynamics	Prof. Amit Patil	63
Sana Khan	DBMS Lab	Prof. Rahul Mehta	84
Sana Khan	Operating Systems	Prof. Vikram Joshi	77

The status bar at the bottom of the pgAdmin window shows: Total rows: 13 Query complete 00:00:00.072.

Figure 10 – Q9 executed in pgAdmin 4 Query Tool, connection college_db/postgres@PostgreSQL 17. pgAdmin status bar: “Total rows: 13 Query complete 00:00:00.072”. Rows returned: 13.

7.11 Q10: Five-table JOIN – Toppers with department and faculty

SQL EXECUTED

```
-- Q10. 5-table JOIN : toppers (marks >= 85) with department and faculty
SELECT s.student_name, d.department_name, c.course_name,
       f.faculty_name, e.marks
FROM student s
INNER JOIN department d ON s.department_id = d.department_id
INNER JOIN enrollment e ON s.student_id = e.student_id
INNER JOIN course c     ON e.course_id   = c.course_id
INNER JOIN faculty f     ON c.faculty_id  = f.faculty_id
WHERE e.marks >= 85
ORDER BY e.marks DESC;
```

ACTUAL OUTPUT – psql 17 (text)

student_name	department_name	course_name	faculty_name	marks
Meera Iyer	Artificial Intelligence & ML	DBMS Lab	Prof. Rahul Mehta	95
Diya Patel	Artificial Intelligence & ML	DBMS Lab	Prof. Rahul Mehta	92
Aarav Sharma	Artificial Intelligence & ML	DBMS Lab	Prof. Rahul Mehta	88
Diya Patel	Artificial Intelligence & ML	Machine Learning	Dr. Anita Deshmukh	85

(4 rows)

ACTUAL SCREENSHOT • Figure 11

The screenshot shows the pgAdmin 4 Query Tool interface. The Query editor contains the following SQL query:

```
-- Q10. 5-table JOIN : toppers (marks >= 85) with department and faculty
SELECT s.student_name, d.department_name, c.course_name,
       f.faculty_name, e.marks
FROM student s
INNER JOIN department d ON s.department_id = d.department_id
INNER JOIN enrollment e ON s.student_id = e.student_id
INNER JOIN course c     ON e.course_id   = c.course_id
INNER JOIN faculty f     ON c.faculty_id  = f.faculty_id
WHERE e.marks >= 85
ORDER BY e.marks DESC;
```

The Data Output pane shows the results of the query, which are 4 rows of student information:

student_name	department_name	course_name	faculty_name	marks
Meera Iyer	Artificial Intelligence & ML	DBMS Lab	Prof. Rahul Mehta	95
Diya Patel	Artificial Intelligence & ML	DBMS Lab	Prof. Rahul Mehta	92
Aarav Sharma	Artificial Intelligence & ML	DBMS Lab	Prof. Rahul Mehta	88
Diya Patel	Artificial Intelligence & ML	Machine Learning	Dr. Anita Deshmukh	85

Total rows: 4 Query complete 00:00:00.078

Figure 11 – Q10 executed in pgAdmin 4 Query Tool, connection college_db/postgres@PostgreSQL 17. pgAdmin status bar: “Total rows: 4 Query complete 00:00:00.078”. Rows returned: 4.

7.12 Q11: LEFT JOIN + IS NULL – Students not enrolled in any course (anti-join)

SQL EXECUTED

```
-- Q11. LEFT JOIN + IS NULL : students not enrolled in any course
SELECT s.student_id, s.student_name, s.admission_year
FROM student s
LEFT JOIN enrollment e ON s.student_id = e.student_id
WHERE e.enrollment_id IS NULL
ORDER BY s.student_id;
```

ACTUAL OUTPUT – psql 17 (text)

student_id	student_name	admission_year
9	Neha Joshi	2025
10	Yash Kulkarni	2025

(2 rows)

ACTUAL SCREENSHOT • Figure 12

The screenshot displays the pgAdmin 4 interface. The left pane shows the database structure for 'college_db', including tables like 'course', 'department', 'enrollment', 'faculty', and 'student'. The central pane shows the SQL query executed:
1 -- Q11. LEFT JOIN + IS NULL : students not enrolled in any course
2 SELECT s.student_id, s.student_name, s.admission_year
3 FROM student s
4 LEFT JOIN enrollment e ON s.student_id = e.student_id
5 WHERE e.enrollment_id IS NULL
6 ORDER BY s.student_id;
The bottom pane shows the 'Data Output' with two rows:
1 | 9 | Neha Joshi | 2025
2 | 10 | Yash Kulkarni | 2025
The status bar at the bottom indicates 'Total rows: 2 Query complete 00:00:00.093'.

Figure 12 – Q11 executed in pgAdmin 4 Query Tool, connection college_db/postgres@PostgreSQL 17. pgAdmin status bar: “Total rows: 2 Query complete 00:00:00.093”. Rows returned: 2.

7.13 Q12: Multi-table JOIN + GROUP BY – Department-wise summary

SQL EXECUTED

```
-- Q12. Multi-table JOIN + GROUP BY : department-wise summary
SELECT d.department_name,
       COUNT(DISTINCT s.student_id) AS total_students,
       COUNT(e.enrollment_id)      AS enrollments,
       ROUND(AVG(e.marks), 2)       AS avg_marks
FROM department d
LEFT JOIN student s   ON d.department_id = s.department_id
LEFT JOIN enrollment e ON s.student_id   = e.student_id
GROUP BY d.department_id, d.department_name
ORDER BY d.department_id;
```

ACTUAL OUTPUT – psql 17 (text)

department_name	total_students	enrollments	avg_marks
Artificial Intelligence & ML	3	6	88.17
Computer Science	3	5	76.60
Electronics & Telecom	1	1	70.00
Mechanical Engineering	1	1	63.00
Civil Engineering	0	0	(null)

(5 rows)

ACTUAL SCREENSHOT • Figure 13

The screenshot displays the pgAdmin 4 interface. The left pane shows the database structure with 'college_db' selected. The central pane shows the SQL query being executed. The bottom pane shows the 'Data Output' tab with the following results:

	department_name	total_students	enrollments	avg_marks
1	Artificial Intelligence & ML	3	6	88.17
2	Computer Science	3	5	76.60
3	Electronics & Telecom	1	1	70.00
4	Mechanical Engineering	1	1	63.00
5	Civil Engineering	0	0	[null]

The status bar at the bottom indicates: Total rows: 5 Query complete 00:00:00.081

Figure 13 – Q12 executed in pgAdmin 4 Query Tool, connection college_db/postgres@PostgreSQL 17. pgAdmin status bar: “Total rows: 5 Query complete 00:00:00.081”. Rows returned: 5.

8. Explanation of Results

The table compares the number of rows each query returned (from the actual outputs in Section 7) and the reason for that count.

Query	Join type	Rows	Why this many rows
Q1	INNER	8	10 students – 2 with NULL department
Q2	LEFT OUTER	10	8 matched + 2 unmatched students
Q3	RIGHT OUTER	8	7 faculty + 1 department without faculty
Q4	FULL OUTER	8	6 matched + 1 course w/o faculty + 1 faculty w/o course
Q5	SELF (INNER)	4	$C(3,2) + C(2,2) = 3 + 1$ pairs
Q6	CROSS	9	3 students $\times$ 3 courses
Q7	CROSS	1	$10 \times 7 = 70$ counted in one row
Q8	INNER (3 tables)	13	one row per enrollment
Q9	INNER + LEFT (4 tables)	13	one row per enrollment, faculty optional
Q10	INNER (5 tables)	4	5 records $\geq$ 85, 1 lost (no faculty)
Q11	LEFT + IS NULL	2	students with no enrollment
Q12	LEFT (3 tables) + GROUP BY	5	one row per department

Q1: INNER JOIN – Student + Department

8 of the 10 students are returned. Neha Joshi (9) and Yash Kulkarni (10) have `department_id = NULL`; since NULL is never equal to any value, the join condition is not satisfied and an INNER JOIN silently drops them. Department 50 (Civil Engineering) is also absent because no student belongs to it.

Q2: LEFT OUTER JOIN – Student + Department

10 rows are returned – all students. The 8 matched rows are identical to Q1; the 2 extra rows (Neha Joshi, Yash Kulkarni) show `[null]` in `department_name`. $\text{Rows}(\text{LEFT JOIN}) = \text{rows}(\text{INNER JOIN}) + \text{unmatched left rows} = 8 + 2 = 10$.

Q3: RIGHT OUTER JOIN – Faculty + Department

8 rows: the 7 faculty members with their departments, plus one extra row for **Civil Engineering** (established 2025) whose `faculty_name` and `designation` are NULL because no faculty row references department 50. The same result could be written as `department d LEFT JOIN faculty f`.

Q4: FULL OUTER JOIN – Course + Faculty

8 rows = 6 matched pairs + 1 unmatched course + 1 unmatched faculty. **Deep Learning** (U25AMPE304) has `faculty_id = NULL`, so its faculty column is NULL; **Prof. Kavita Rao** (107) is not assigned to any course, so her course columns are NULL. $\text{FULL OUTER JOIN} = \text{LEFT JOIN} \cup \text{RIGHT JOIN}$.

Q5: SELF JOIN – Faculty members in the same department

4 pairs. AI & ML has 3 faculty members, giving $C(3,2) = 3$ pairs; Computer Science has 2, giving 1 pair; Electronics and Mechanical have only one faculty member each, so they produce no pair. Without the `<` condition the result would contain $3^2 + 2^2 + 1 + 1 = 15$ rows (including self-pairs and mirrored pairs).

Q6: CROSS JOIN – Student × Course (AI & ML)

3 AI & ML students (Aarav, Diya, Meera) × 3 AI & ML courses (DBMS Lab, Machine Learning, Deep Learning) = 9 rows. Each student appears once with every course.

Q7: CROSS JOIN – size of the full Cartesian product

student has 10 rows and course has 7 rows, so the Cartesian product has $10 \times 7 = 70$ rows, confirming $|A \times B| = |A| \cdot |B|$. This is why a missing join condition in a real query produces a huge, meaningless result.

Q8: Three-table JOIN – Student → Enrollment → Course

13 rows – exactly one per enrollment record. Students 9 and 10 are not enrolled, so they do not appear. The output shows the student name, the course name and the marks, which are stored in three different tables.

Q9: Four-table JOIN – Student → Enrollment → Course → Faculty

13 rows, same as Q8. For Meera Iyer's Deep Learning enrollment the faculty is shown as **Not Assigned**. Had the last join been an INNER JOIN, that row would have been dropped and only 12 rows returned.

Q10: Five-table JOIN – Toppers with department and faculty

4 rows. Five enrollments actually have marks ≥ 85 (95, 92, 90, 88, 85), but Meera Iyer's Deep Learning record (90) is missing because its course has no faculty and every join here is an INNER JOIN. This shows that each INNER JOIN in a chain can eliminate rows – an outer join (as in Q9) is needed when optional relationships must be kept.

Q11: LEFT JOIN + IS NULL – Students not enrolled in any course (anti-join)

2 rows: Neha Joshi and Yash Kulkarni, both admitted in 2025, have not enrolled in any course. The same result can be obtained with NOT EXISTS or NOT IN.

Q12: Multi-table JOIN + GROUP BY – Department-wise summary

5 rows, one per department. AI & ML: 3 students, 6 enrollments, average $(88+79+92+85+95+90)/6 = 88.17$. Computer Science: 3 students, 5 enrollments, average $383/5 = 76.60$. Civil Engineering appears with 0 students and a NULL average only because LEFT JOINs were used; COUNT(column) ignores NULLs, which is why it shows 0 and not 1.

Overall observations

- INNER JOIN returned 8 students while LEFT JOIN returned 10: the difference is exactly the rows whose foreign key is NULL. Outer joins never return fewer rows than the corresponding inner join.
- RIGHT JOIN on faculty–department exposed a department with no staff; FULL OUTER JOIN exposed unmatched rows on both sides at once.
- The SELF JOIN needed two aliases and a non-equality condition ($<$) to avoid self-pairs and duplicate pairs.
- The CROSS JOIN result size (70) equalled the product of the table sizes (10×7).
- In multi-table joins every INNER JOIN in the chain can remove rows (Q10 lost one record); a LEFT JOIN keeps optional relationships (Q9).

9. Result

Multi-table join queries were successfully implemented and executed on the **college_db** College Management System database in PostgreSQL 17. INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, SELF and CROSS joins were demonstrated, along with 3-, 4- and 5-table joins, an anti-join and a join with GROUP BY. The actual outputs were verified, and it was observed that inner joins return only matching rows, outer joins preserve unmatched rows with NULLs, a self join compares rows of the same table, and a cross join returns the Cartesian product.

10. Viva Questions and Answers

Q1. What is a join? Why is it needed?

Ans. A join combines rows from two or more tables based on a related column. It is needed because a normalised database stores related facts (student, department, course) in separate tables to avoid redundancy.

Q2. What is the difference between INNER JOIN and OUTER JOIN?

Ans. INNER JOIN returns only rows satisfying the join condition. OUTER JOIN (LEFT/RIGHT/FULL) also returns unmatched rows of one or both tables, filling the missing columns with NULL.

Q3. Are LEFT JOIN and RIGHT JOIN interchangeable?

Ans. Yes. *A RIGHT JOIN B* gives the same rows as *B LEFT JOIN A*; only the default column order differs. LEFT JOIN is used more often for readability.

Q4. What is a SELF JOIN? Why are aliases compulsory in it?

Ans. A self join joins a table with itself, e.g. finding faculty of the same department. Since the same table name appears twice, aliases (f1, f2) are required to tell the two copies apart.

Q5. What is a CROSS JOIN? How many rows does it produce?

Ans. It is the Cartesian product of two tables with no join condition. It produces $\text{rows}(A) \times \text{rows}(B)$ rows ($10 \times 7 = 70$ for student and course). It is useful for generating all combinations.

Q6. What happens if the join condition is omitted in a multi-table query?

Ans. A Cartesian product is produced, giving a very large and mostly meaningless result.

Q7. What is the difference between equi-join, theta join and natural join?

Ans. An equi-join uses only "=" in the condition; a theta join may use any comparison (<, >, <>); a natural join is an equi-join on all columns with the same name, and the common column appears once.

Q8. Why is NATURAL JOIN considered risky?

Ans. It joins automatically on every same-named column. If a new column with the same name is added later (e.g. `created_at`), the join condition changes silently.

Q9. What is the difference between ON and USING?

Ans. ON allows any condition and different column names. USING (col) is shorthand when both tables have a column of the same name and returns that column only once.

Q10. Does the position of a filter (ON vs WHERE) matter in an outer join?

Ans. Yes. A condition in ON is applied while matching, so unmatched rows are still kept. The same condition in WHERE is applied after the join and removes NULL-extended rows, effectively turning it into an inner join.

Q11. How do you find students who have not enrolled in any course?

Ans. Use an anti-join: student LEFT JOIN enrollment ... WHERE enrollment.enrollment_id IS NULL (or NOT EXISTS).

Q12. How many join conditions are required to join n tables?

Ans. At least $n - 1$ join conditions. For example, the 5-table join Q10 uses 4 ON conditions.

Q13. What is FULL OUTER JOIN? Is it supported in every DBMS?

Ans. It returns all rows from both tables, matched where possible. PostgreSQL, Oracle and SQL Server support it; MySQL does not, and it is emulated there using LEFT JOIN UNION RIGHT JOIN.

Q14. Which join algorithms does PostgreSQL use?

Ans. Nested Loop Join, Hash Join and Merge Join. The planner chooses one using table statistics and indexes; EXPLAIN shows the chosen plan.

Q15. What is the difference between a join and a subquery?

Ans. A join combines columns from several tables into one result row. A subquery is a query nested inside another, often used for filtering. Many subqueries can be rewritten as joins, which the optimiser often runs more efficiently.

Appendix A – Screenshot Verification Log

Generated by `verify_screenshots.py`. "Editor = SQL" means the text in the pgAdmin editor at capture time equals the SQL printed in this record. "Grid = psql" means every cell of the pgAdmin result grid equals the row returned by psql for the same SQL. The SHA-256 fingerprints identify the exact image files in the `screenshots/` folder.

Fig.	Query	File	Rows	Editor = SQL	Grid = psql	SHA-256 (first 16 hex)
1	Q0	Q0.png	1	Yes	Yes	e608fec238f1ae01
2	Q1	Q1.png	8	Yes	Yes	f2c5559796ce41df
3	Q2	Q2.png	10	Yes	Yes	1087a8683740ca48
4	Q3	Q3.png	8	Yes	Yes	05495df8f9cc7e0d
5	Q4	Q4.png	8	Yes	Yes	84a9f11ffaa94389
6	Q5	Q5.png	4	Yes	Yes	371bcdfcc150bbce
7	Q6	Q6.png	9	Yes	Yes	adb3085318e300f7
8	Q7	Q7.png	1	Yes	Yes	8ac7d61ae7624221
9	Q8	Q8.png	13	Yes	Yes	142b79c9d1c9fab5
10	Q9	Q9.png	13	Yes	Yes	8df6794cd3a98f08
11	Q10	Q10.png	4	Yes	Yes	45d3e360cbfacc0f
12	Q11	Q11.png	2	Yes	Yes	32a416dd02279b49
13	Q12	Q12.png	5	Yes	Yes	62f7ed666bc9b206

Capture environment

Item	Value
DBMS server	PostgreSQL 17.10 (x86_64 Linux)
GUI client	pgAdmin 4 v9.18, Query Tool (desktop mode, rendered in Chromium)
CLI client	psql 17
Database / user	college_db / postgres
Capture date (UTC)	2026-10-06